

What this project did, in plain language

October 2026

The question

Turbulent fluids spread their energy across big swirls and small swirls in a known way. This project checks whether neural networks do something similar inside their layers.

“Energy” here is not physics energy stored in the air. It is the size of the network’s numbers: square the weights or the activations, then average or add them up. A *scale* is the size of the thing that number sits on: a patch of pixels, a mesh edge on the Earth, or a wavelength.

The measurement, used everywhere

Same four steps in every notebook.

1. Pick the scales.
 - Image nets: how many pixels a layer can see. Call that R . The matching wavenumber is $k = 1/R$. A big receptive field is a small k .
 - GraphCast: each level of its icosahedral mesh, from about 10,000 km edges down to 156 km.
 - FourCastNet: Fourier wavenumber inside the model, $k = \sqrt{k_x^2 + k_y^2}$.
2. Compute an energy E at each scale: mean or sum of squares of the weights, or of the activations.
3. Turn those energies into fractions that add up to 1,

$$p_k = \frac{E(k)}{\sum E}, \quad S_H = - \sum p_k \log_2 p_k.$$

S_H is the hydrodynamic entropy. $S_{\max} = \log_2(\text{number of scales})$ is the value if every scale held the same energy. The ratio $S_H/S_{\max}$ near 1 means the energy is spread out. A small ratio means it is piled onto a few scales.

4. Fit a straight line on a log–log plot of energy versus scale. The slope is α in $E \propto k^\alpha$.

The slopes we compare against:

- Kolmogorov turbulence: $\alpha = -5/3 \approx -1.67$.
- Large weather systems (Nastrom–Gage, scales above about 1000 km): about -3 .
- Smaller weather systems (about 100–1000 km): about -1.67 .

Two ways of adding up the squares are both used, and they give different slopes.

- **Mean per weight or per edge.** Fair. A fine mesh has far more edges; dividing by that count removes the head count.
- **Raw sum.** The fine mesh wins just because it has more edges. A steep positive slope from a raw sum is usually that head-count effect.

Image networks

These are the warm-up. Same measurement, on models everyone already knows.

Pretrained AlexNet, VGG-16, VGG-19. Mean of W^2 per convolution, plotted against receptive field. Energy sits in the first convolution. Normalized entropy: AlexNet 0.57, VGG-16 0.29, VGG-19 0.28. The fit was written as $\text{mean}(W^2) \sim (1/R)^\beta$ with β about 1.0 to 1.1. Read that as: mean weight energy drops as the layer sees a larger patch.

Same VGG models, but the sum of W^2 , not the mean. The sum grows toward deeper layers, so the slope flips sign and becomes shallow: VGG-16 $\alpha = -0.51$, VGG-19 $\alpha = -0.41$. Entropy is high ($S_H/S_{\max}$ about 0.94 and 0.96): the *totals* are spread fairly evenly even though the *averages* are not. A 10,000-sample bootstrap on VGG-19 puts α in $[-0.47, -0.35]$. Local slopes jump around, so one straight line is a rough summary.

Train a VGG-style net from scratch and record the slope every few epochs. α goes from -0.67 at the start to about -0.48 by epoch 30.

Train a small MLP (5 hidden layers, 512 units) on CIFAR-10. Activation slope flattens during training: about -3 at random initialization, about -1 after 30 epochs in the first run. A second run with error bars: activation α from -1.67 to -0.93 ; weight α from -0.60 to -1.22 ; weight entropy from 0.91 down to about 0.67. Five layers is too few for a serious power-law test. The plots show the trend. They do not prove a power law.

Pretrained ResNet-50, five stages. Activation slope $\alpha = -0.06$ with $R^2 = 0.004$: flat, no trend. Weight slope $\alpha = -0.77$ with $R^2 = 0.96$, entropy ratio 0.74. Skip connections keep the activation size from dying as you go deeper. The weights still get smaller at finer scales.

GraphCast

GraphCast is DeepMind’s weather model. It is a graph network on a mesh of the Earth. Three public checkpoints:

Model	Grid	Mesh	Edges
Small	1°, 13 levels	<i>M0–M5</i>	81,900
Operational	0.25°, 13 levels	<i>M0–M6</i>	327,660
Full	0.25°, 37 levels	<i>M0–M6</i>	327,660

Each one has hidden size 512 and 16 rounds of message passing.

Approach 1: geometry only, no weather

Do not forecast anything. Take the first linear layer of the mesh-edge encoder. It is one shared matrix W , shape 4×512 . Each edge is a 4-number vector: the three components of “receiver minus sender”, plus the length. Multiply, square, and average inside each mesh level. That average is E .

Result, almost the same for all three models:

Model	S_H	$S_H/S_{\max}$	α	R^2
Small	1.205	47%	-1.87	0.9995
Operational	1.206	43%	-1.85	0.9986
Full	1.206	43%	-1.84	0.9986

About 70% of this energy is on the coarsest mesh, *M0*. The slope is a bit steeper than -1.67 . This is the encoder looking at edge lengths. It is not a weather state.

Approach 2: run the model on real weather

Load GraphCast Small. Run one forecast step on sample ERA5 data. Record the edge features inside the network after message passing. Use the *mean* energy per edge. (An older notebook summed the squares and got $\alpha \approx +1.72$. $M5$ has 1,024 times as many edges as $M0$, so the sum mostly counts edges. The mean is the number used below.)

Split each level’s energy into two pieces:

- S : energy of the average activation (the shared pattern);
- V : the spread around that average.

On the six mesh levels:

Piece	Slope α	95% range	R^2
Total	−0.28	[−0.56, −0.03]	0.70
Signal S	−0.60	[−1.15, −0.12]	0.70
Variance V	+0.16	[+0.04, +0.26]	0.79

Binning edges by their true length, instead of by mesh level, moves the total slope only to about −0.33.

Checks that were actually run:

- Shuffle the mesh, or replace features with noise. The slope goes to about 0. The −0.28 shows up when the trained weights see the real mesh.
- A break near 544 km fits better than one straight line. Coarse side $\alpha \approx -0.04$, fine side $\alpha \approx -0.85$.
- Fourier spectra of the *forecast fields* (temperature, pressure, 10 m wind) have slopes about −3.1 to −4.2. The weather coming *out* of the model is steep. The internal edge features are shallow.
- A Clauset test on the list of individual edge energies prefers a lognormal (and similar curves) over a power-law histogram. That is a test of the histogram, separate from the slope of mean energy versus scale.
- Stack every mesh level at every message-passing step: 204 points. Pooled slope $\alpha = -0.06$, 95% range [−0.11, −0.01], $R^2 = 0.025$. A weak drop is detectable. It is not −1.67, and a straight line does not describe the cloud of points.
- Energy moves toward small scales. The variance moves the other way. The variance’s entropy ratio is about 0.99: that part is spread across scales.

FourCastNet

FourCastNet is NVIDIA’s weather model. It works in Fourier space, in 12 blocks, on a 0.25° grid, 20 weather variables, hidden width 1024.

What was built:

1. Load the official AFNO network.
2. Attach hooks so each block’s internal activation is copied out during a forward pass. No training, no backward pass.
3. For every frequency, compute $k = \sqrt{k_x^2 + k_y^2}$.

4. Add $|a|^2$ into rings of constant k . That ring sum is the shell energy $E(k)$.
5. From $E(k)$, compute S_H in each block and the slope.

The saved run used fake input, not a real ERA5 file, and not a finished pass on the pretrained forecast weights. Measured slope: about $+0.98$. White noise *must* give about $+1$ after you bin it into rings, because a bigger ring contains more frequencies. Entropy barely moved from block to block (changes under 0.001 bits). So the binning code is doing the geometry correctly, and the network does not invent large-scale structure in pure noise.

Still not done: the same measurement on real ERA5 with the pretrained weights. The expectation written in the report: inside a block the slope may be much flatter than the atmospheric -3 , and across the 12 blocks S_H may fall as the forecast gets organized. Do this on activations. The weights themselves are copied onto every wavenumber, so a spectrum of the weights alone is flat by construction.

The short version

1. VGG piles mean weight energy into early layers. ResNet’s skip connections keep activation energy flat. Training an MLP flattens a steep random activation spectrum toward a slope near -1 .
2. GraphCast’s edge-geometry encoder, with no weather input, has slope about -1.85 on all three checkpoints, close to $-5/3$, with most of that energy on the biggest edges.
3. GraphCast’s internal activations on a real forecast step have slope about -0.28 . The drop is in the average pattern. The spread around that average stays flat. Shuffle the mesh and the slope disappears.
4. The forecast fields GraphCast writes out have steep spectra, slopes about -3 to -4 .
5. FourCastNet’s measuring code is checked on noise (slope $+1$, flat entropy). The real-weather run is the remaining piece.